

Wardhook for VS Code

Catch personal data in your AI prompts before it ever leaves your laptop.

Teams building LLM agents paste real data into prompts all day long. A customer's claim, a support transcript, a row from production used to debug a failing case. That text gets committed, logged, and sent to a model provider. By the time anyone asks *"what personal data went through this agent?"*, the answer is scattered across a hundred files.

Wardhook for VS Code finds that data while you are still typing it. It underlines national insurance numbers, card numbers, API keys and the rest directly in the editor, and replaces them with a safe placeholder in one click. Nothing is uploaded. There is no account and no server.

What it does

- **Live warnings.** Personal data is underlined as you type, the same way a typo is.
- **One-click cleanup.** Replace a finding with a label such as [US_SSN], or clean a whole file at once.
- **A findings list** for the whole project, so you can see every file at a glance.
- **It talks to GitHub Copilot.** Ask @wardhook questions in plain English, and Copilot can run its seven tools by itself.
- **Four ready-made rule sets** for general use, insurance, healthcare and finance, or bring your own.

```
11
12 CLAIM_PROMPT = """Customer Dana Reyes filed claim CLM-8891.
13 SSN on file: 796-30-8562. Card ending 4111 1111 1111 1111.
14 Contact: dana.reyes@acme-insure.com or 415-555-0142.
15 Summarise the claim for the adjuster."""
16
17 AWS_ACCESS_KEY_ID = "AKIAIOSFODNN7EXAMPLE"
18
```

Three findings underlined in a real file. Notice line 14: the email and phone number were detected too, but deliberately not shown. That choice is explained on page 2.

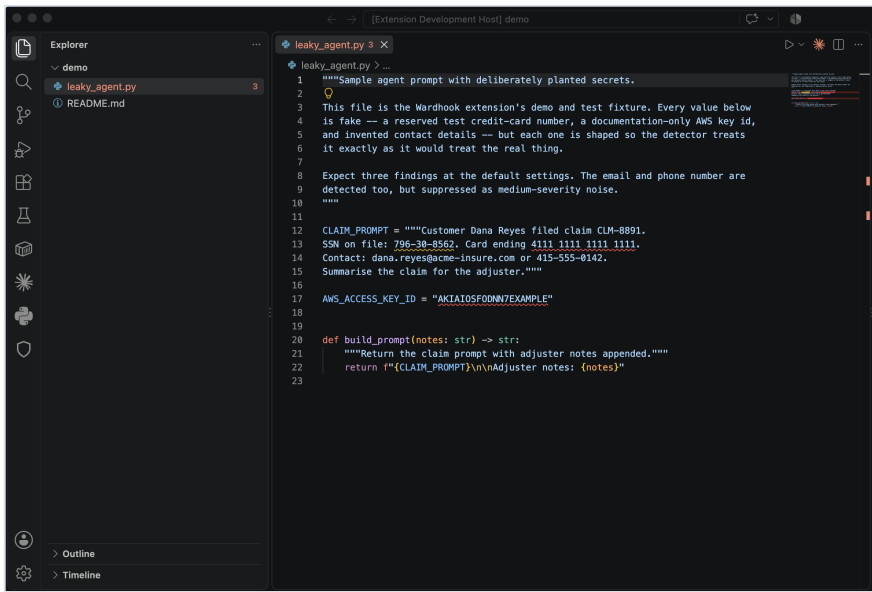
The part people remember

The detector never hands over the secret it found. It reports *where* something is, never *what* it is. So the extension can draw a warning over a credit card number that it was never actually given. There is an automated test that proves it: scan a card number, and the card number does not appear anywhere in the result.

Why it is useful to a team building agents

Four situations that come up constantly once an agent handles real customer data:

Situation	What Wardhook does
You paste a real customer record into a prompt to debug it	Underlines it immediately, before you commit. One click swaps it for a placeholder and the prompt still works.
A teammate asks for help and you want to share a file	Redact Entire File gives you a copy that is safe to paste into Slack or a ticket.
Compliance asks what personal data your agent can touch	Scan Workspace produces a count for every file, grouped by type and risk.
Your agent reads documents that users upload	Scores that text for prompt-injection attempts, such as "ignore all previous instructions".



The extension running. The file and its tab both show a count of 3, and the three real findings are underlined while the two low-risk ones stay out of your way.

It is tuned to be quiet, and that was the hard part

Any tool can flag every number that looks like a card. That tool gets switched off in a day. The real work was making it worth leaving on.

We pointed the detector at the Wardhook project itself: **89 matches across 66 files**. Almost all were harmless. Example values in documentation. 127.0.0.1 in a config file. Fake data inside tests, which is exactly where fake data belongs. Only 10 matches were confirmed by a real checksum, and every one of those mattered.

Setting	Warnings shown	Verdict
Show everything	89	Unusable. People turn it off.
Only high risk and above	42	Still too noisy.
The shipped default	8	Quiet, and missed nothing real.
Only checksum-confirmed	10	Safest, but misses API keys.

The default shows a finding if a checksum proves it is real (a card number that passes the Luhn check), **or** if it is high risk by nature (a national insurance number, an AWS key). Test folders are skipped, because sample data there is intentional. **That removes about 91% of the noise without losing a single real leak.** One setting turns everything back on when you want it.

It works inside GitHub Copilot

Type @wardhook in Copilot Chat and ask for what you want. You can use a direct command such as /scan, or just ask a question and let Copilot pick the right tool on its own.

Tool	What Copilot can do with it
wardhook_scan_pii	Find personal data in a file or a snippet.
wardhook_redact	Return a clean copy with placeholders.
wardhook_scan_workspace	Summarise the whole project.
wardhook_score_injection	Score text for prompt-injection attempts.
wardhook_list_packs	List the rule sets and what each one detects.
wardhook_explain_entity	Explain why something was, or was not, flagged.
wardhook_check_tool_policy	Check whether an agent role may call a given tool.

Your code never leaves the machine

- **No server, no account, no telemetry.** The checking runs on your own computer.
- **Nothing is uploaded** for analysis, not even to Wardhook. There is nowhere to upload to.
- **Signing in to GitHub is optional** and used for exactly one thing: downloading a shared rule file your team keeps in a private repository. That traffic only ever comes inward.
- **The tool never receives the secret** it warns you about, as described on page 1.

Fast enough to leave switched on

Measurement	Result	What it means for you
Checking a file as you type	12 ms	Faster than you notice. It keeps up with typing.
Starting up	About 0.16 s, once	Paid a single time when the editor opens a file.
Scanning in bulk	About 3 MB per second	A large project scans in seconds.

The speed comes from one design decision: the checker is a single small helper program that starts once and stays running, rather than being launched again for every keystroke. Starting it costs 0.16 seconds; asking it a question costs 12 milliseconds.

How to install it

You need **VS Code 1.95 or newer** and **Python 3.10 or newer**. That is the whole list. GitHub Copilot is optional, and everything except the chat features works without it.

1 Install the extension file

Open VS Code. Press `Cmd+Shift+P` (or `Ctrl+Shift+P` on Windows), type **Install from VSIX**, and choose `wardhook-vscode-0.1.0.vsix`.

2 Open any file with some text in it

A Python file, a Markdown note, a YAML config, a log. The shield icon appears in the bar on the left.

3 Say yes if it offers to finish setting up

The first time it runs, it looks for a Python that can do the checking. If it cannot find one, it offers a button to install the small package it needs. Click it and wait a few seconds. Nothing else is required.

4 Try it

Type a fake card number such as `4111 1111 1111 1111` into a file. It should be underlined straight away. Put your cursor on it and press `Cmd+.` to replace it with a placeholder.

If nothing happens

Open the command palette and run **Wardhook: Show Log**. It prints which Python it chose and why. The most common cause is an old Python: on macOS the built-in one is version 3.9, which is too old, so the extension deliberately skips it and looks for a newer one.

Settings worth knowing

Setting	Default	What it changes
Pack	default	Which rule set to use: general, insurance, healthcare or finance.
Min severity	high	Lower it to medium to see emails and phone numbers too.
Validated only	off	Turn on to show only findings proven by a checksum.
Exclude globs	tests	Folders to leave alone. Test data is usually fake on purpose.
Scan on type	on	Turn off to check only when you save.
Python path	auto	Pin a specific Python if you want to.

Wardhook is open source under the MIT licence. The extension is a front end for `wardhook-guardrails`, published on PyPI, which depends on a single library and is small enough to run inside a Lambda or an air-gapped container. The wider project also covers cost and token tracking per agent step, and regression testing for agent behaviour.

Questions people ask

Question	Answer
Will it slow my editor down?	No. A check takes about 12 milliseconds and only runs once you pause typing.
Does it catch everything?	No, and it does not claim to. Measured against a labelled test set it finds 84.3% of personal data, and 94.2% of what it flags is genuinely personal data. Those numbers are published in the project, not estimated. It is a safety net, not a guarantee.
Can we add our own rules?	Yes. Rules are a plain YAML file. A team can keep one in a private repository and everyone's editor will use the same definitions.
Do we need GitHub Copilot?	No. Copilot only adds the chat features. Warnings, fixes and scanning work on their own.
Is any code sent anywhere?	No. There is no server to send it to.
What if it flags something harmless?	Use the quick fix <i>Stop reporting this type</i> , or add the folder to the ignore list. Test folders are already ignored.

How it works, in one paragraph

The extension itself is TypeScript and does no detecting. It starts one small Python helper when you open a file and keeps it running for the session. Every time you pause typing, it sends the text down a pipe and gets back a list of positions. That is the whole design, and it is why the editor stays responsive: the expensive part, loading the rules, happens once instead of on every keystroke.